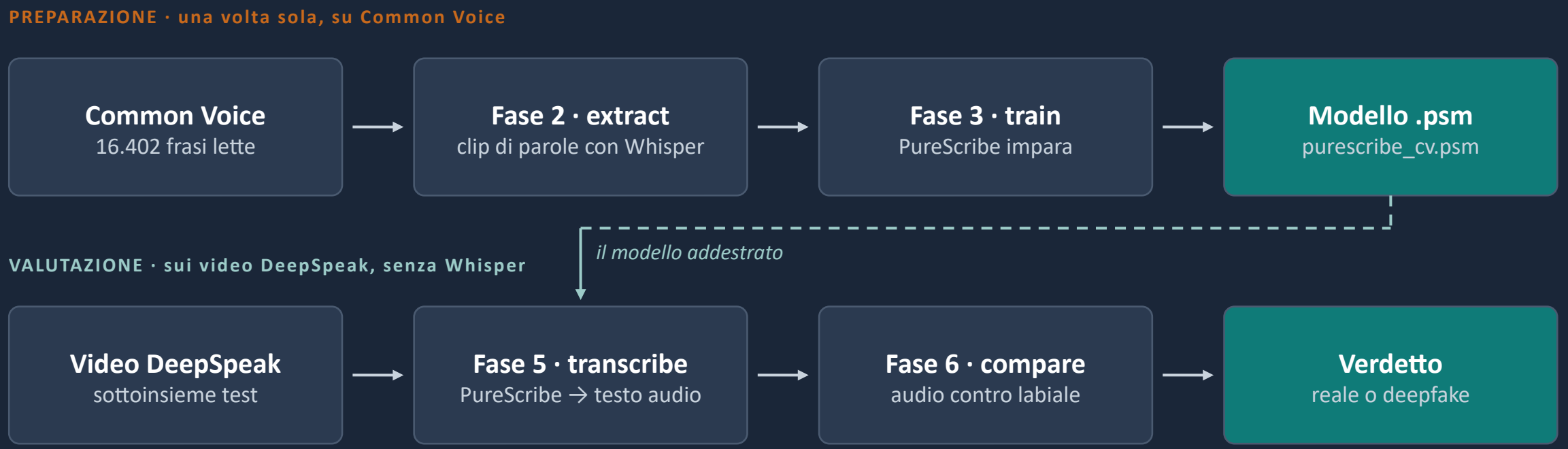
La pipeline della tesi con PureScribe

PureScribe prende il posto di Whisper in “Lost in Translation”: impara le parole da Common Voice, poi trascrive i video di DeepSpeak



Le fasi 1 e 4 (divisione e validazione su Common Voice) sono state tolte: tutto Common Voice serve ad addestrare, la prova si fa su DeepSpeak.

Il punto di partenza: Lost in Translation

IL METODO DELLA BASELINE · BOHACEK E FARID, CVPRW 2024



Un video vero dice quello che si sente. Il metodo trascrive due volte lo stesso video: dall'audio e leggendo le labbra. In un video autentico i due testi si somigliano; in un lip-sync deepfake la bocca non segue le parole dell'audio.

Nella tesi cambia solo chi ascolta. Al posto di Whisper c'è PureScribe; video, testi del labiale, pulizia dei testi, distanza e soglia restano quelli della baseline.

Cosa viene riusato. Le trascrizioni del labiale e quelle Whisper della baseline, prodotte il 20/09/2026, vengono solo lette: sui video non si riesegue né Whisper né la lettura labiale.

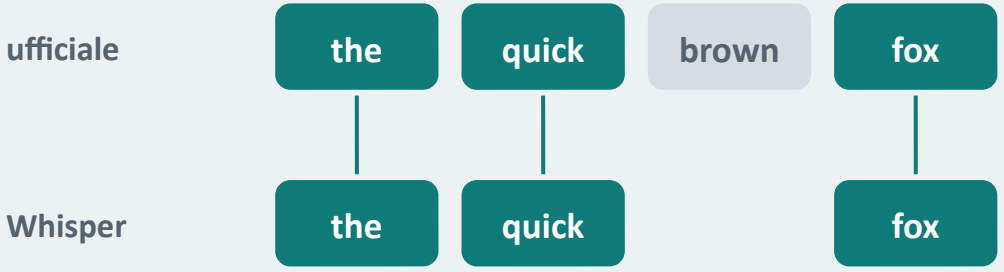
Fase 2 · Ritagliare le parole da Common Voice

2 commonvoice_purescribe.py extract · l'unico punto in cui gira Whisper

Il materiale di partenza. Le 16.402 registrazioni di Common Voice in corpus\dev: persone che leggono una frase di cui conosciamo il testo ufficiale. PureScribe però impara da parole singole, quindi serve sapere dove inizia e finisce ogni parola.

Whisper ascolta, ma non decide l'etichetta. Riceve solo l'audio e restituisce le parole che sente con i loro tempi. Si tengono solo le parole identiche al testo ufficiale, nello stesso ordine; il nome del clip è sempre la parola ufficiale.

Ogni parola trovata diventa un file. Un ritaglio con un piccolo margine (50 ms prima, 120 ms dopo), salvato nella cartella che porta il nome della parola.



→ clip salvati: the, quick, fox · brown viene ignorata

```
# commonvoice_purescribe.py · extract()
result = model.transcribe(audio, language="en",
                          word_timestamps=True)
matches = dict(match_word_indices(
    [token for token, _ in words],          # testo ufficiale
    [word["word"] for word in recognized]))  # parole Whisper
wav = output / label / f"{Path(name).stem}_w{index:02d}.wav"
save_wav(wav, audio[first:last], TARGET_SR)
```

Fase 2 · Le regole del corpus

2 commonvoice_purescribe.py extract · regole e tracciabilità

Il vocabolario nasce dai dati

Nessuna lista decisa prima: entra ogni parola fatta di lettere, con ' o - solo in mezzo, così we’re resta diversa da were.

Né troppi né troppo pochi

Al massimo 23 clip per parola, così the e and non dominano; alla fine le parole con meno di 12 clip vengono tolte.

Tante voci diverse

Le registrazioni sono lette in un ordine mescolato con un seme fisso (20261014): i clip di una parola non vengono da un solo lettore.

Quando una parola viene scartata

Whisper non l’ha sentita uguale al testo, i suoi tempi mancano o sono impossibili, il ritaglio dura meno di 100 ms, oppure la parola ha già 23 clip. I tempi non vengono mai corretti né inventati: la parola si salta.

```
# cosa scrive la fase 2 nella cartella di output
<parola>\<audio>_w<indice>.wav   # mono, 16 kHz
clips.jsonl       # una riga per ogni parola vista
extraction.json   # parametri, vocabolario, conteggi
# una riga reale di clips.jsonl
{"audio": "common_voice_en_28123723.mp3",
 "word_index": 12, "official_word": "back.",
 "label": "back", "status": "quota_reached"}
```

Fase 3 · Dai clip ai numeri

3 `python -m purescribe.cli train <cartella della fase 2> -o <modello>.psm`

Il comando legge una cartella per parola. Il nome della cartella è l'etichetta: l'addestramento non sa nulla di Whisper o di Common Voice, vede solo i file prodotti dalla fase 2.

Ogni clip viene ripulito. Ogni file è portato a una sola traccia a 16 kHz con volume uniformato; il rilevatore di voce toglie il silenzio rimasto ai bordi. Un po' di silenzio viene messo da parte per insegnare al modello com'è lo sfondo.

Poi diventa numeri. Ogni 10 ms un pezzetto di 25 ms è descritto da 39 numeri sul timbro e su come cambia. La loro taratura è calcolata una volta su tutti i clip e salvata nel modello.

```
# train.py · load_word_corpus(): una cartella = una parola
for word_dir in sorted(...):
    for clip in clips:
        wave = load_audio(clip, rate=feature_config.rate)
        samples = trim_silence(wave.samples, wave.rate)
        f = mfcc(samples, feature_config)
    corpus[word_dir.name] = feats

# train.py · train(): una taratura per tutti i clip
stats = estimate_stats([...])
corpus = {w: [apply_stats(f, stats) for f in feats] ...}

# features.py · mfcc(): 25 ms ogni 10 ms -> 39 numeri
frames = frame_signal(preemphasis(x, cfg.preemph), ...)
log_mel = np.log(mel_energy)                  # scala mel
cepstra = dct(log_mel, ...)[:, : cfg.n_cepstra]
d1 = delta(cepstra)                           # come cambia
```

Fase 3 · Un modello per ogni parola

3 train.py · hmm.py → purescribe_cv.psm

una parola



i tratti si percorrono in ordine, senza tornare indietro

```
# train.py · train()
for word in sorted(corpus):
    n_states = choose_n_states(feats, tcfg)  # tratti
    model = HMM(n_states, dim, tcfg.n_mixtures, name=word, ...)
    model.fit(feats, n_iter=tcfg.iterations, method=tcfg.method)
# cli.py: scrive in una cartella temporanea, poi rinomina
staged = rec.save(Path(staging) / "model.psm")
staged.rename(out)
```

Una catena di tratti

Ogni parola è una sequenza di tratti di suono; in ognuno si resta finché il suono dura, così va bene sia chi parla piano sia chi parla in fretta. Le parole lunghe hanno più tratti.

Impara correggendosi

Prima divide ogni esempio in parti uguali, una per tratto; poi riallinea gli esempi, ricalcola come suona ogni tratto e ripete, al massimo 10 volte.

Il risultato: purescribe_cv.psm

Il modello usato nella valutazione conosce 1676 parole più il silenzio. Il file contiene anche le impostazioni di ascolto e di ricerca.

Fase 5 · Trascrivere i video DeepSpeak

5 lost_in_translation.py transcribe · nessun Whisper

1 **Quali video**
Quelli che la baseline aveva trascritto in entrambi i rami: dai suoi file si leggono solo i nomi, cercati poi in D:\DeepSpeak\test.

2 **Estrarre l'audio**
ffmpeg tira fuori la traccia audio del video, a una sola traccia e 16 kHz, come per i clip di training.

3 **Trascrivere**
Il riconoscitore ascolta l'audio e restituisce il testo, fatto solo di parole del suo vocabolario.

4 **Scrivere una riga**
Video, esito, testo e impronta del modello finiscono in deepfake_purescribe.tsv, una riga per video.

```
# lost_in_translation.py · transcribe()
videos = list(baseline_videos(args.baseline_transcripts,
                              args.subset)[0])
model_hash = sha256(model_path)
recognizer = Recognizer.load(model_path)
for position, video in enumerate(videos[len(done):], ...):
    wave = load_audio(path, rate=recognizer.feature_config.rate)
    text = recognizer.transcribe(wave).text
    writer.writerow([video, status, text, detail, model_hash])
```

UNA RIGA REALE DEL TSV

real/227-12476.mp4 · ok

"running company girl large there just could call america rock"

Nel file attuale: 120 video con esito ok e 4 non più presenti su disco, sui 124 trascritti dalla baseline.

Fase 5 · Come PureScribe trova la frase

5 recognizer.py · decoder.py · dentro recognizer.transcribe()

```
# recognizer.py · transcribe()
use_vad = wave.duration > self.vad_threshold_s    # 30 s
result = self.decoder.decode(self.features(wave.samples))

# decoder.py · decode(): un passo ogni 10 ms
for t in range(1, T):
    exit_score, exit_state = self._segment_max(
        delta + self._log_exit)      # chi finisce una parola
    logb = self._emissions_at(X[t], ...)
    new_delta = best + logb          # avanza nelle parole
    links.extend(...)                # annota i cambi di parola
    delta = self._prune(delta)       # scarta chi è indietro
return self._traceback(delta, bp, links, T)
```

a **Tutte le parole in cerchio**
Le 1676 parole e la pausa sono collegate ad anello: dopo una parola ne può venire qualsiasi altra, senza grammatica.

b **Molte ipotesi in parallelo**
Mentre l'audio scorre porta avanti tante frasi possibili e scarta subito quelle troppo improbabili.

c **Alla fine, la migliore**
Ripercorre all'indietro l'ipotesi migliore e legge le parole attraversate; le pause non vengono scritte.

d **Audio brevi in un colpo solo**
Fino a 30 s l'audio è cercato in un solo passaggio; oltre, il rilevatore di voce lo divide nei tratti parlati.

Fase 6 · Il confronto con il labiale

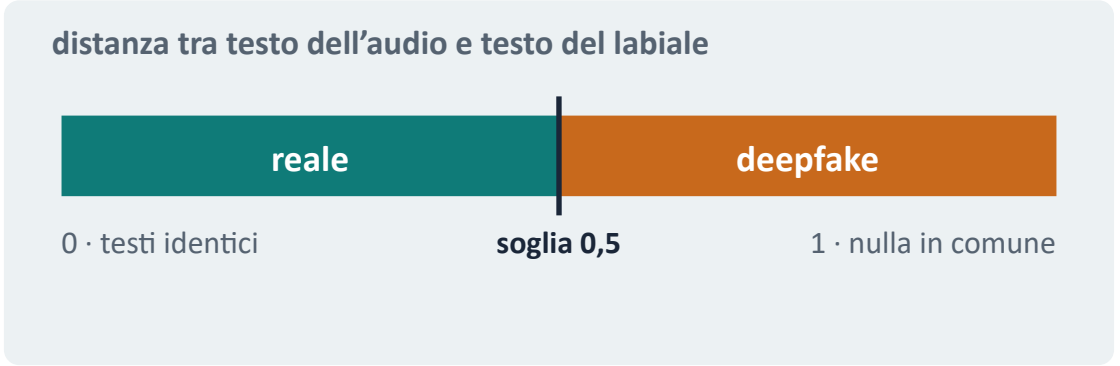
6 lost_in_translation.py compare

Gli stessi video per tutti e due. Entrano solo i video che PureScribe ha trascritto con esito ok, e su quegli stessi video si ricalcola anche Whisper, dai testi salvati dalla baseline.

Le stesse regole della baseline. Il testo di PureScribe viene pulito come la baseline puliva quello di Whisper: minuscole, niente punteggiatura né esitazioni come um o uh. Poi si contano le lettere da cambiare per passare da un testo all'altro, divise per la lunghezza del più lungo.

Oltre 0,5 è deepfake. La soglia è quella della baseline e non viene ritoccata per PureScribe.

Cosa esce: deepfake_comparison.json, con veri e falsi positivi e negativi, accuratezza, F1 e distanze per metodo, per Whisper e per PureScribe.




```
# lost_in_translation.py · compare()
for system, audio in (("whisper", whisper_text),
        ("purescribe", standardize_text(pure_text))):
    distance = normalized_levenshtein(audio, video_text)
    prediction = "fake" if distance > threshold else "real"

# normalized_levenshtein(): uguale alla baseline
round(edit_distance(first, second)
      / max(len(first), len(second)), 4)
```

Cosa tenere presente

V

Conosce solo le sue parole

1676 parole: nel parlato libero molte parole sono fuori vocabolario, quindi anche nei video autentici il testo audio si allontana dal labiale.

≠

Due mondi acustici

Impara da parole ritagliate da frasi lette di Common Voice e lavora sull'audio registrato in webcam dei video DeepSpeak.

?

Tagli non ascoltati

Il confronto con il testo ufficiale dice che la parola c'è, non che il taglio sia perfetto: i clip non sono stati ascoltati uno per uno.

n

Un banco di prova piccolo

120 video, 20 autentici e 100 deepfake, con un solo metodo lip-sync (latentsync); la soglia resta quella della baseline.

Ogni miglioramento si prova come esperimento separato, senza scegliere parametri guardando DeepSpeak.